The following cells demonstrate basic matrix operations such as addition, scalar multiplication, transpose, matrix multiplication and solving system of linear equations

In [30]:
```
import numpy as np

def generate_random_uniform_matrix(m, n):
    """
    Generates a random matrix of size m x n with
    entries from a uniform distribution between 0 and 1.
    """
    matrix = np.random.rand(m, n)
    return matrix

# Example usage of the function
m_dim = 3
n_dim = 5
A = generate_random_uniform_matrix(m_dim, n_dim)
B = generate_random_uniform_matrix(m_dim, n_dim)

print(f"Generated a {m_dim}x{n_dim} random matrix A:")
print(A)

print(f"Generated a {m_dim}x{n_dim} random matrix B:")
print(B)

# Addition
mat_sum = A + B
print("\nResult of Matrix Addition (A + B):\n", mat_sum)

# Scalar multiplication
mat_scalar_mult = 3*A
print("\nResult of scalar Matrix 3A:\n", mat_scalar_mult)

# Transpose of A
A_t = A.T
print("\nResult of Matrix Transpose A^T:\n", A_t)

# Matrx multiplication A^T*B
mat_mult = A_t @ B
print("\nResult of Matrix multiplication A^T*B:\n", mat_mult)

# selecting a submatrix from A
columns=[0,1,3]
C=A[:,columns]
print("\nResult of Sub Matrix extraction C=A(:,[1, 2, 4]):\n", C)
print(C.shape)

# Generating a random vector of length 3
y = generate_random_uniform_matrix(m_dim, 1)
print(f"Random vector y:\n", y)
#solving Cx=y
x_solution = np.linalg.solve(C, y)
print(f"Solution vector x:\n", x_solution)
```

```
Generated a 3x5 random matrix A:
[[0.2502429  0.54922666 0.71459592 0.66019738 0.2799339 ]
 [0.95486528 0.73789692 0.55435405 0.61172075 0.41960006]
 [0.24773099 0.35597268 0.75784611 0.01439349 0.11607264]]
Generated a 3x5 random matrix B:
[[0.04600264 0.0407288  0.85546058 0.70365786 0.47417383]
 [0.09783416 0.49161588 0.47347177 0.17320187 0.43385165]
 [0.39850473 0.6158501  0.63509365 0.04530401 0.37461261]]

Result of Matrix Addition (A + B):
 [[0.29624554 0.58995547 1.57005651 1.36385524 0.75410773]
 [1.05269944 1.22951279 1.02782582 0.78492262 0.85345171]
 [0.64623572 0.97182278 1.39293976 0.0596975  0.49068526]]

Result of scalar Matrix 3A:
 [[0.75072869 1.64767999 2.14378777 1.98059213 0.83980169]
 [2.86459584 2.21369075 1.66306216 1.83516224 1.25880019]
 [0.74319297 1.06791804 2.27353833 0.04318047 0.34821792]]

Result of Matrix Transpose A^T:
 [[0.2502429  0.95486528 0.24773099]
 [0.54922666 0.73789692 0.35597268]
 [0.71459592 0.55435405 0.75784611]
 [0.66019738 0.61172075 0.01439349]
 [0.2799339  0.41960006 0.11607264]]

Result of Matrix multiplication A^T*B:
 [[0.20365225 0.63218418 0.82350707 0.35269304 0.62573166]
 [0.2393142  0.60435699 1.04529111 0.53039977 0.71391856]
 [0.38911333 0.76835349 1.35508289 0.63317966 0.86324882]
 [0.09595388 0.33648491 0.86354655 0.57115633 0.58383636]
 [0.10018442 0.28916677 0.5118582  0.27491176 0.35826378]]

Result of Sub Matrix extraction C=A(:,[1, 2, 4]):
 [[0.2502429  0.54922666 0.66019738]
 [0.95486528 0.73789692 0.61172075]
 [0.24773099 0.35597268 0.01439349]]
(3, 3)
Random vector y:
 [[0.62585992]
 [0.50313626]
 [0.85648984]]
Solution vector x:
 [[-1.13553454]
 [ 3.24988834]
 [-1.32521878]]
```

In the following cell, I have demonstrated the code for computing row reduced Echelon form a given matrix. Consider A to be any matrix of size $m \times n$. Generate a random matrix of integers of size $m \times n$. Then using the in-built commands, compute the row reduced Echelon form this randomly geberated matrix. In this example, you can try to input the specific matrix $A = \left[\begin{matrix} 1 & 2 & -1 & 3 \\ 3 & -2 & -1 & 1 \\ 4 & 0 & -2 & 4 \end{matrix} \right]$ and calculate its row reduced Echelon form. Deduce rank (A).

In [5]:
```
import numpy as np
import sympy as sp
```

```python
# 1. Generate a random matrix using NumPy

# This creates a 3x4 matrix with random integers between -10 and 10
random_array = np.random.randint(-10, 11, size=(3, 4))
# This creates a 3x4 matrix with random numbers between -10 and 10
# random_array = np.random.uniform(-10, 11, size=(2, 4))

print("--- Original Random Matrix ---")
print(random_array)
print("\n" + "="*30 + "\n")

# 2. Convert the NumPy array to a SymPy Matrix for perfect fraction math
sympy_matrix = sp.Matrix(random_array)

# 3. Perform Row Reduced Echelon Form (RREF)
# .rref() returns a tuple: (the RREF matrix, the indices of pivot columns
rref_matrix, pivot_columns = sympy_matrix.rref()

print("--- Row Reduced Echelon Form (RREF) ---")
sp.pprint(rref_matrix) # pprint makes fractions look beautiful

print(f"\nPivot Columns located at indices: {pivot_columns}")
```

```
--- Original Random Matrix ---
[[ 6  0  8 -1]
 [ 1 10 -6  9]
 [-6 -8  6  7]]

==============================

--- Row Reduced Echelon Form (RREF) ---
⎡          -287 ⎤
⎢1  0  0  ──────⎥
⎢           122 ⎥
⎢               ⎥
⎢           517 ⎥
⎢0  1  0   ─── ⎥
⎢           244 ⎥
⎢               ⎥
⎢           100 ⎥
⎢0  0  1   ─── ⎥
⎣           61  ⎦

Pivot Columns located at indices: (0, 1, 2)
```

In the following cell, I have demonstrated a code for generating a data matrix for the Least Squares Problem. The relationship $y=f(x)$ is an unknown relationship between the input data $x \in \mathbb{R}^n$ and the output data or labels $y \in \mathbb{R}$ that we are trying to find out. The idea is to choose basis functions $f_1, f_2, \ldots,f_k : \mathbb{R}^{n} \rightarrow \mathbb{R}$ and try to approximate $f(x) \approx \alpha_1 f_1(x) + \alpha_2 f_2(x) + \cdots + \alpha_k f_k(x)$. We call the linear combination of basis functions as the model and $\alpha_1,\ldots,\alpha_k$ are the model parameters. The objective is to compute the model parameters such that the model approximates the unknown function f in the best possible way.

Let $\mathcal{D} = \{ (x_i,y_i) \}_{i=1}^N$ be the historical data that is available. We want to approximate f using the choice of the model mentioned above by finding the best

choice of model parameters $\alpha_1,\ldots,\alpha_k$ using the evidence $\mathcal{D}$. Define $\mathbf{\alpha} = \left[\begin{matrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_k \end{matrix}\right] \in \mathbb{R}^k$. We can define the residuals r_i as $$ r_i = y_i - \hat{y}_i = y_i - \left(f_1(x_i) \alpha_1 + \cdots + f_k(x_i) \alpha_k \right) = y_i - \left[\begin{matrix} f_1(x_i) & f_2(x_i) & \cdots & f_k(x_i) \end{matrix} \right] \left[\begin{matrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_k \end{matrix}\right]~~\textrm{ for } i=1,2,\ldots,N $$ Clearly we want residuals to be as small as possible. Therefore, we minimize $\sum_{i=1}^N r_i^2$ with respect to the unknown model parameters $\alpha \in \mathbb{R}^k$. Let $\mathbf{r} = \left[\begin{matrix} r_1 \\ r_2 \\ \vdots \\ r_N \end{matrix} \right] \in \mathbb{R}^N$ be the vector of the residuals (often called as residual vector), ${\mathbf{y}} = \left[\begin{matrix} {y}_1 \\ {y}_2 \\ \vdots \\ {y}_N \end{matrix} \right] \in \mathbb{R}^N$ be the vector of labels and $\hat{\mathbf{y}} = \left[\begin{matrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_N \end{matrix} \right] \in \mathbb{R}^N$ be the vector of predictions. $$ \sum_{i=1}^N r_i^2 = \mathbf{r}^{\top} \mathbf{r} =\left(\mathbf{y} - \hat{\mathbf{y}} \right)^{\top} \left(\mathbf{y} - \hat{\mathbf{y}} \right) = \left(\mathbf{y} - \mathbf{\Phi} \mathbf{\alpha} \right)^{\top} \left(\mathbf{y} - \mathbf{\Phi} \mathbf{\alpha} \right) = \Vert \mathbf{y} - \mathbf{\Phi} \mathbf{\alpha} \Vert_2^2 $$ where $$ \mathbf{\Phi} = \left[\begin{matrix} f_1(x_1) & f_2(x_1) & \cdots & f_k(x_1) \\ f_1(x_2) & f_2(x_2) & \cdots & f_k(x_2) \\ \vdots & \vdots & \vdots & \vdots \\ f_1(x_N) & f_2(x_N) & \cdots & f_k(x_N) \end{matrix} \right] \in \mathbb{R}^{N \times k} ~~\textrm{ and } \hat{\mathbf{y}} = \mathbf{\Phi} \mathbf{\alpha} $$ In the following example, we construct $\mathbf{\Phi}$, $\mathbf{y}$ and $\hat{\mathbf{y}}$ and solve the LS problem using the in-built command.

The first data set is $\mathcal{D} = \{(x_i,y_i)\}_{i=1}^{100}$ where $x_i,y_i \in \mathbb{R}$ and $f_1=1$, $f_2 = t$, \ldots, $f_k=t^{k-1}$ are the polynomials chosen as a basis function. Then we are trying to fit a degree $k-1$ polynomial to the data $\mathcal{D}$. Here $n=1$, the number of parameters to be estimated is k. (Here we have generated y from the relationship $y=3+2t-3t^2+4t^3$). Use different values of k and see what happens.

In the second example we use the same input data points x_i, but the labels here are different y_i corrupted by noise. We perform the same analysis.

In the third case labels are generated using the function $y=tsin(1/t) + t^2$. Use different values of k and see what happens?

Since all these examples can be visualized in $\mathbb{R}^2$, we have plotted the data and the estimated function to visualize how good the fit is.

In [8]:
```
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate historical data D = {(t_i, y_i)}
N = 100
t = np.random.uniform(0, 1, N)  # Uniformly distributed inputs over [0, 1
y = 3 + 2 * t - 3 * (t**2) + 4 * (t**3)
```

```
# 2. Try different numbers of basis functions k (degree = k - 1)
k_values = [2, 4, 6]  # Degree 1 (linear), Degree 3 (true degree), Degree

plt.figure(figsize=(12, 4))

# Fine grid for smooth visualization of fitted model without sorting orig
t_fine = np.linspace(0, 1, 200)

for idx, k in enumerate(k_values):
    # Construct Design Matrix Phi in R^(N x k) for training data
    Phi = np.column_stack([t**j for j in range(k)])

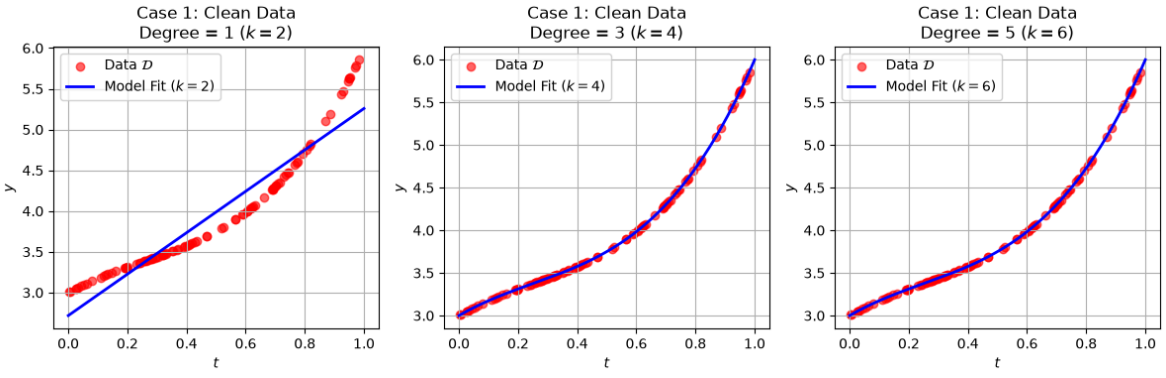
    # Solve Least Squares: min ||y - Phi * alpha||_2^2
    alpha, _, _, _ = np.linalg.lstsq(Phi, y, rcond=None)

    # Design matrix for smooth visualization grid
    Phi_fine = np.column_stack([t_fine**j for j in range(k)])
    y_fine_hat = Phi_fine @ alpha

    # Plot results
    plt.subplot(1, len(k_values), idx + 1)
    plt.scatter(t, y, color="red", alpha=0.6, label=r"Data $\mathcal{D}$"
    plt.plot(
        t_fine,
        y_fine_hat,
        color="blue",
        linewidth=2,
        label=rf"Model Fit ($k={k}$)",
    )
    plt.title(f"Case 1: Clean Data\nDegree = {k-1} ($k={k}$)")
    plt.xlabel("$t$")
    plt.ylabel("$y$")
    plt.legend()
    plt.grid(True)

plt.tight_layout()
plt.show()
```

```
Matplotlib is building the font cache; this may take a moment.
```



In [9]:

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate noisy historical data D = {(t_i, y_i)}
np.random.seed(42)  # For reproducible noise
N = 100
t = np.random.uniform(0, 1, N)
noise = np.random.normal(0, 0.2, N)
y = 3 + 2 * t - 3 * (t**2) + 4 * (t**3) + noise
```

```
# 2. Fit models with different values of k
k_values = [2, 4, 15]  # Underfit (k=2), Good fit (k=4), Overfit (k=15)

plt.figure(figsize=(12, 4))
t_fine = np.linspace(0, 1, 200)

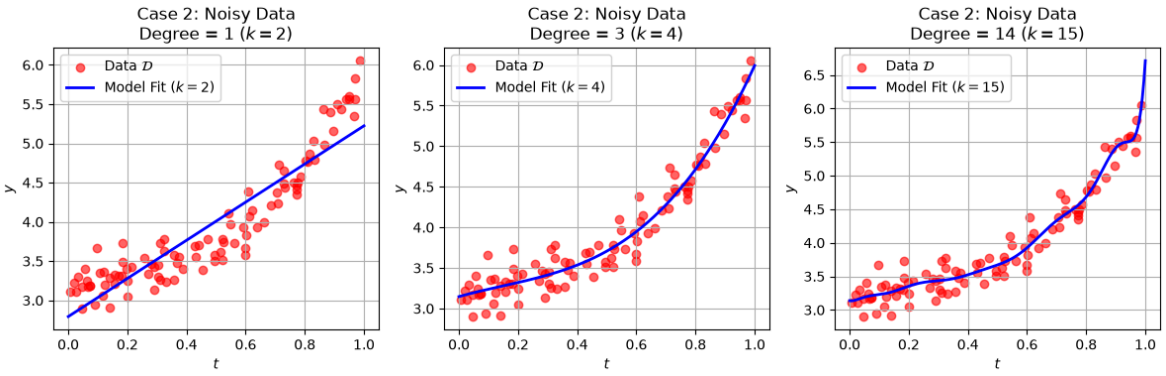
for idx, k in enumerate(k_values):
    # Construct Design Matrix Phi in R^(N x k)
    Phi = np.column_stack([t**j for j in range(k)])

    # Solve Least Squares
    alpha, _, _, _ = np.linalg.lstsq(Phi, y, rcond=None)

    # Evaluate fitted model on visualization grid
    Phi_fine = np.column_stack([t_fine**j for j in range(k)])
    y_fine_hat = Phi_fine @ alpha

    # Plot results
    plt.subplot(1, len(k_values), idx + 1)
    plt.scatter(t, y, color="red", alpha=0.6, label=r"Data $\mathcal{D}$"
    plt.plot(
        t_fine,
        y_fine_hat,
        color="blue",
        linewidth=2,
        label=rf"Model Fit ($k={k}$)",
    )
    plt.title(f"Case 2: Noisy Data\nDegree = {k-1} ($k={k}$)")
    plt.xlabel("$t$")
    plt.ylabel("$y$")
    plt.legend()
    plt.grid(True)

plt.tight_layout()
plt.show()
```



In [10]:

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Generate data for non-polynomial function
N = 100
# Avoid t=0 to prevent zero division in sin(1/t)
t = np.random.uniform(0.01, 1, N)
y = t * np.sin(1 / t) + t**2

# 2. Fit polynomial approximations of varying complexity
k_values = [3, 6, 12]
```

```
plt.figure(figsize=(12, 4))
t_fine = np.linspace(0.01, 1, 300)

for idx, k in enumerate(k_values):
    # Construct Design Matrix Phi in R^(N x k)
    Phi = np.column_stack([t**j for j in range(k)])

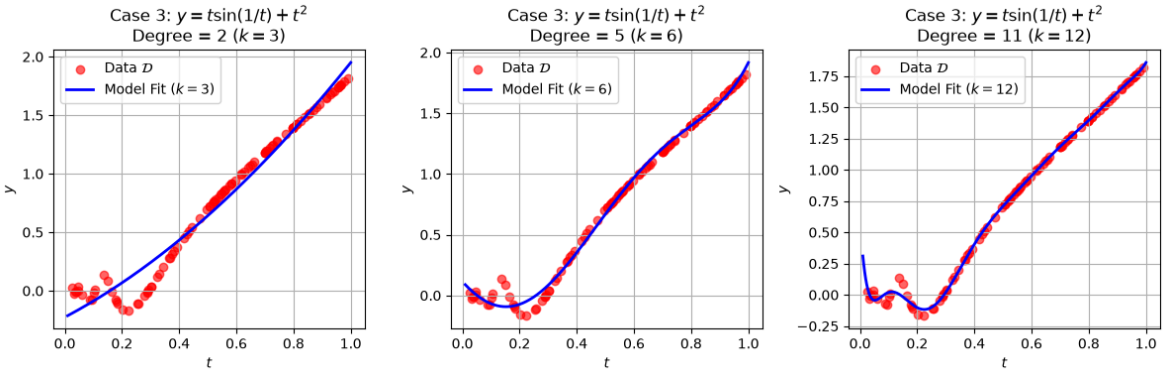
    # Solve Least Squares
    alpha, _, _, _ = np.linalg.lstsq(Phi, y, rcond=None)

    # Evaluate fitted model on visualization grid
    Phi_fine = np.column_stack([t_fine**j for j in range(k)])
    y_fine_hat = Phi_fine @ alpha

    # Plot results
    plt.subplot(1, len(k_values), idx + 1)
    plt.scatter(t, y, color="red", alpha=0.6, label=r"Data $\mathcal{D}$"
    plt.plot(
        t_fine,
        y_fine_hat,
        color="blue",
        linewidth=2,
        label=rf"Model Fit ($k={k}$)",
    )
    plt.title(f"Case 3: $y = t\sin(1/t) + t^2$\nDegree = {k-1} ($k={k}$)"
    plt.xlabel("$t$")
    plt.ylabel("$y$")
    plt.legend()
    plt.grid(True)

plt.tight_layout()
plt.show()
```

```
<>:37: SyntaxWarning: invalid escape sequence '\s'
<>:37: SyntaxWarning: invalid escape sequence '\s'
/var/folders/rm/jc_n1_ns561g_91f28gfp_dm0000gn/T/ipykernel_2858/140729442
3.py:37: SyntaxWarning: invalid escape sequence '\s'
  plt.title(f"Case 3: $y = t\sin(1/t) + t^2$\nDegree = {k-1} ($k={k}$)")
```



In [11]:

```
import numpy as np
import matplotlib.pyplot as plt

def fit_and_plot_polynomial(t, y, degrees, title_prefix):
    """
    Fits polynomial models of different numbers of basis functions k (deg
    and plots the original data along with the fitted curves.
    """
    plt.figure(figsize=(12, 4))
```

```
    # Grid for smooth line rendering without altering or sorting original
    t_fine = np.linspace(t.min(), t.max(), 200)

    for idx, k in enumerate(degrees):
        # 1. Construct Design Matrix Phi in R^(N x k) using original unso
        # Phi = [1, t, t^2, ..., t^(k-1)]
        Phi = np.column_stack([t**j for j in range(k)])

        # 2. Solve Least Squares: min ||y - Phi * alpha||_2^2
        alpha, residuals, rank, s = np.linalg.lstsq(Phi, y, rcond=None)

        # 3. Predict on grid for visual display: y_hat = Phi_fine * alpha
        Phi_fine = np.column_stack([t_fine**j for j in range(k)])
        y_fine_hat = Phi_fine @ alpha

        # 4. Plot results
        plt.subplot(1, len(degrees), idx + 1)
        plt.scatter(t, y, color='red', alpha=0.6, label='Data points $\ma
        plt.plot(t_fine, y_fine_hat, color='blue', linewidth=2, label=f'F
        plt.title(f'{title_prefix}\nPolynomial Degree = {k-1} ($k={k}$)')
        plt.xlabel('$t$')
        plt.ylabel('$y$')
        plt.legend()
        plt.grid(True)

    plt.tight_layout()
    plt.show()

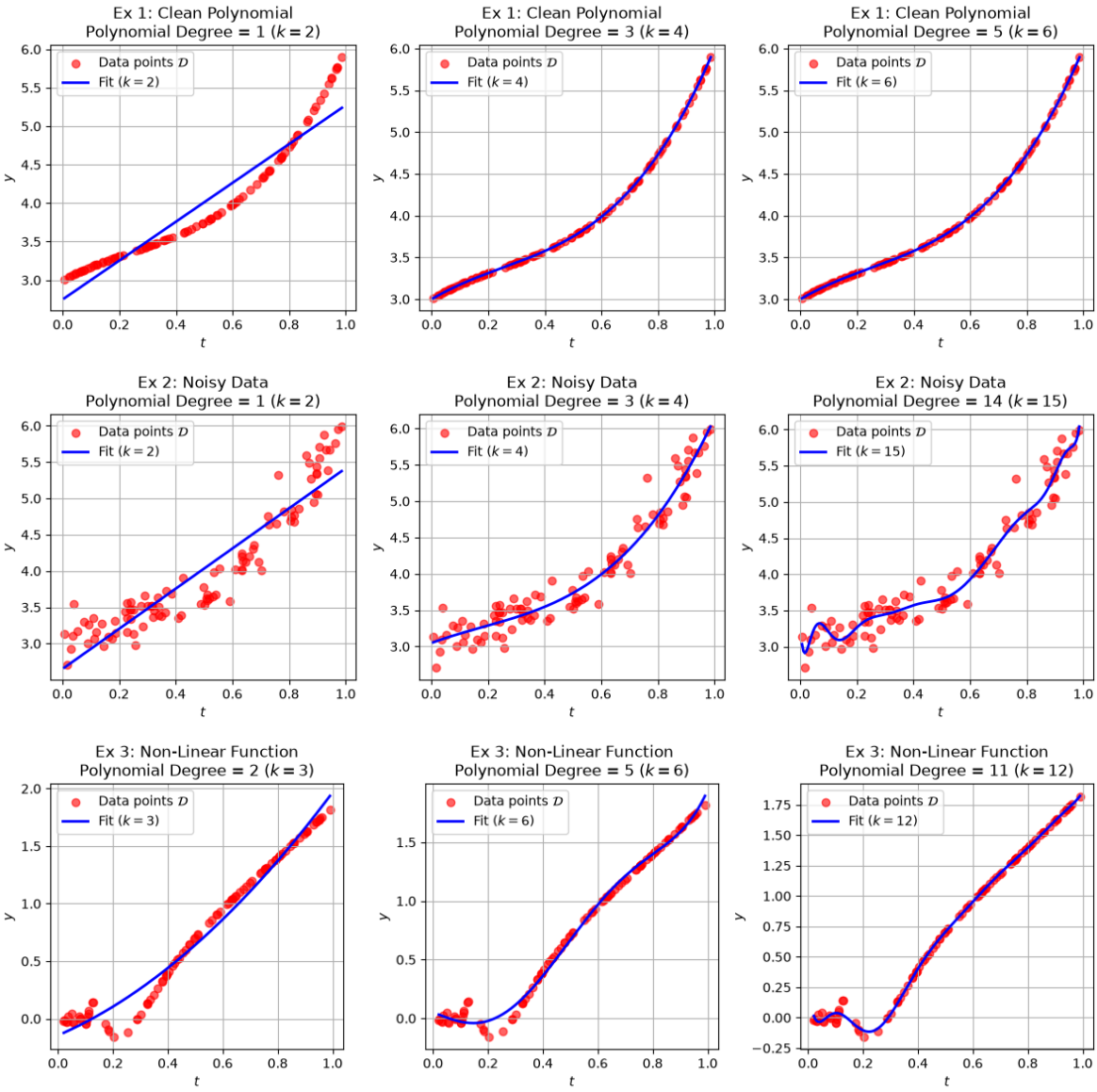
# ------------------------------------------------------------------------
# Data Setup (N=100 points, t ~ Uniform(0, 1))
# ------------------------------------------------------------------------
np.random.seed(42)
N = 100

# Case 1: Clean Polynomial Data
t1 = np.random.uniform(0, 1, N)
y1 = 3 + 2 * t1 - 3 * (t1**2) + 4 * (t1**3)
fit_and_plot_polynomial(t1, y1, degrees=[2, 4, 6], title_prefix="Ex 1: Cl

# Case 2: Noisy Data
t2 = np.random.uniform(0, 1, N)
noise = np.random.normal(0, 0.2, N)
y2 = 3 + 2 * t2 - 3 * (t2**2) + 4 * (t2**3) + noise
fit_and_plot_polynomial(t2, y2, degrees=[2, 4, 15], title_prefix="Ex 2: N

# Case 3: Non-Polynomial Function y = t*sin(1/t) + t^2
t3 = np.random.uniform(0.01, 1, N)
y3 = t3 * np.sin(1 / t3) + t3**2
fit_and_plot_polynomial(t3, y3, degrees=[3, 6, 12], title_prefix="Ex 3: N
```

```
<>:28: SyntaxWarning: invalid escape sequence '\m'
<>:28: SyntaxWarning: invalid escape sequence '\m'
/var/folders/rm/jc_n1_ns561g_91f28gfp_dm0000gn/T/ipykernel_2858/315000187
3.py:28: SyntaxWarning: invalid escape sequence '\m'
  plt.scatter(t, y, color='red', alpha=0.6, label='Data points $\mathcal
{D}$')
```



Gaussian Radial Basis Functions (RBFs)

A **Radial Basis Function (RBF)** is a real-valued function whose output depends solely on the distance between the input point x and a fixed center point c. Because the value depends only on the Euclidean distance $r = \|x - c\|_2$, the function is symmetric in all directions.

The standard **Gaussian Radial Basis Kernel** is defined as:

$$ \phi(x) = \exp\left(-\frac{\|x - c\|^2}{2\sigma^2} \right) $$

Key Parameters

- **Center ($c \in \mathbb{R}^n$):** The point where the basis function reaches its maximum peak at 1.0.
- **Width ($\sigma > 0$):** Controls the spatial spread of the kernel. Larger σ creates broader, smoother basis functions, while smaller σ creates narrow, localized peaks.

Model Approximation via RBF Linear Combination

When using k Radial Basis Functions to approximate an unknown target function $\hat{y} = f(x)$, the prediction is computed as a linear combination of kernel outputs:

$$ \hat{y}(x) = \sum_{j=1}^{k} \alpha_j \phi_j(x) = \sum_{j=1}^{k} \alpha_j \exp\left(-\frac{\|x - c_j\|^2}{2\sigma_j^2} \right) $$

In matrix form for a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, the prediction vector $\hat{\mathbf{y}} \in \mathbb{R}^N$ is:

$$ \hat{\mathbf{y}} = \mathbf{\Phi} \mathbf{\alpha} $$

where the design matrix $\mathbf{\Phi} \in \mathbb{R}^{N \times k}$ is given by:

$$ \mathbf{\Phi} = \begin{bmatrix} \exp\left(-\frac{\|x_1 - c_1\|^2}{2\sigma_1^2}\right) & \exp\left(-\frac{\|x_1 - c_2\|^2}{2\sigma_2^2}\right) & \cdots & \exp\left(-\frac{\|x_1 - c_k\|^2}{2\sigma_k^2}\right) \\ \exp\left(-\frac{\|x_2 - c_1\|^2}{2\sigma_1^2}\right) & \exp\left(-\frac{\|x_2 - c_2\|^2}{2\sigma_2^2}\right) & \cdots & \exp\left(-\frac{\|x_2 - c_k\|^2}{2\sigma_k^2}\right) \\ \vdots & \vdots & \ddots & \vdots \\ \exp\left(-\frac{\|x_N - c_1\|^2}{2\sigma_1^2}\right) & \exp\left(-\frac{\|x_N - c_2\|^2}{2\sigma_2^2}\right) & \cdots & \exp\left(-\frac{\|x_N - c_k\|^2}{2\sigma_k^2}\right) \end{bmatrix} $$

The optimal parameters $\mathbf{\alpha}^* = [\alpha_1, \alpha_2, \ldots, \alpha_k]^\top \in \mathbb{R}^k$ are calculated by solving Ordinary Least Squares:

$$ \mathbf{\alpha}^* = \arg\min_{\mathbf{\alpha}} \|\mathbf{y} - \mathbf{\Phi}\mathbf{\alpha}\|_2^2 = (\mathbf{\Phi}^\top \mathbf{\Phi})^{-1} \mathbf{\Phi}^\top \mathbf{y} $$

In [14]:
```python
import numpy as np
import matplotlib.pyplot as plt

# 1. Define evaluation domain
x = np.linspace(-5, 5, 500)

# 2. Set up RBF centers and width (sigma)
centers = [-3, -1, 1, 3]
sigma = 0.8

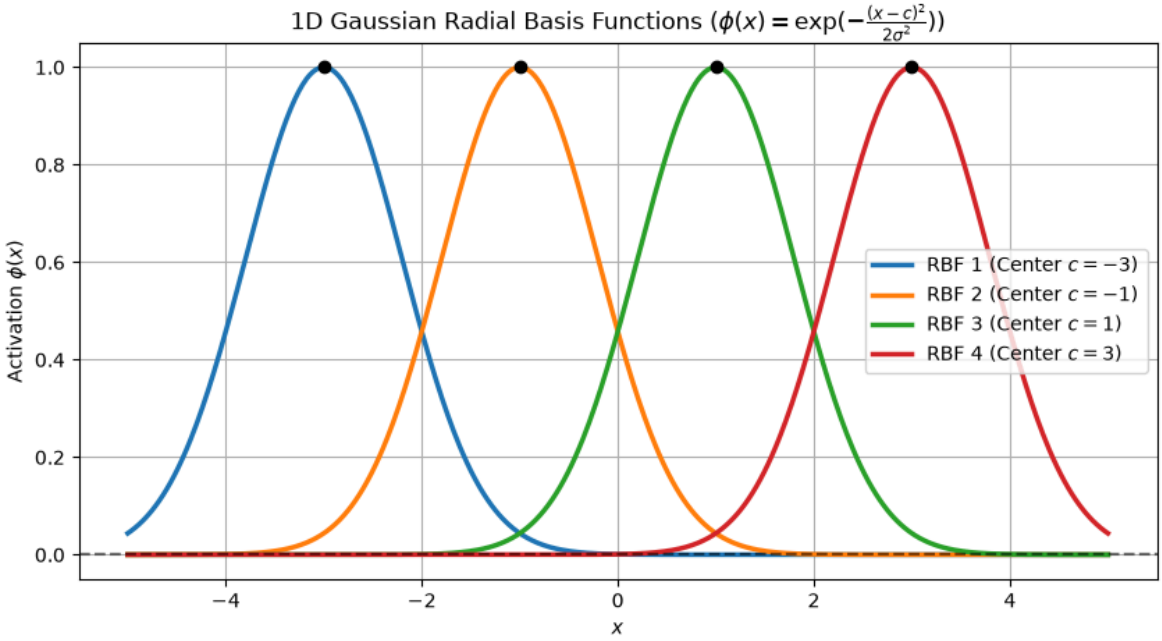
plt.figure(figsize=(10, 5))

# 3. Plot individual RBF basis functions
for i, c in enumerate(centers):
    # Gaussian RBF formula: exp(- (x - c)^2 / (2 * sigma^2))
    rbf = np.exp(-((x - c) ** 2) / (2 * sigma**2))
    plt.plot(x, rbf, label=f"RBF {i+1} (Center $c={c}$)", linewidth=2.5)
    plt.scatter(c, 1.0, color="black", zorder=5)  # Mark center peaks

plt.title(
    r"1D Gaussian Radial Basis Functions ($\phi(x) = \exp(-\frac{(x-c)^2}
    fontsize=12,
)
plt.xlabel("$x$")
plt.ylabel("Activation $\phi(x)$")
plt.axhline(0, color="black", linestyle="--", alpha=0.5)
```

```
plt.legend()
plt.grid(True)
plt.show()
```

```
<>:25: SyntaxWarning: invalid escape sequence '\p'
<>:25: SyntaxWarning: invalid escape sequence '\p'
/var/folders/rm/jc_n1_ns561g_91f28gfp_dm0000gn/T/ipykernel_2858/306758033.
py:25: SyntaxWarning: invalid escape sequence '\p'
  plt.ylabel("Activation $\phi(x)$")
```



In [19]:

```
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------------------
# 1. Generate Training Data D = {(t_i, y_i)} over (0, 1]
# ------------------------------------------------------------------------
np.random.seed(42)
N = 100
# Uniform random sampling over [0.01, 1] without sorting
t = np.random.uniform(0.01, 1.0, N)
y = t * np.sin(1 / t) + t**2

# Dense grid for continuous visualization without modifying training data
t_fine = np.linspace(0.01, 1.0, 500)

# ------------------------------------------------------------------------
# 2. Configure Non-Uniform RBF Centers (Clustered near 0, Sparser near 1)
# ------------------------------------------------------------------------
k_values = [6, 15, 25]  # Compare model performance with varying number o

plt.figure(figsize=(14, 4))

for idx, k in enumerate(k_values):
    # Logarithmic distribution of centers: dense near 0.01, sparse near 1
    centers = np.logspace(np.log10(0.01), np.log10(1.0), k)

    # Adaptive width (sigma): proportional to adjacent center distances
    sigmas = np.zeros(k)
    if k > 1:
        sigmas[0] = (centers[1] - centers[0]) * 1.5
```

```
        sigmas[-1] = (centers[-1] - centers[-2]) * 1.5
        sigmas[1:-1] = (centers[2:] - centers[:-2]) * 0.75
    else:
        sigmas = np.array([0.5])

    # ------------------------------------------------------------------
    # 3. Construct Design Matrix Phi in R^(N x k) for Training Data
    # ------------------------------------------------------------------
    # Phi_ij = exp( - (t_i - c_j)^2 / (2 * sigma_j^2) )
    Phi = np.column_stack([
        np.exp(-((t - centers[j]) ** 2) / (2 * sigmas[j] ** 2))
        for j in range(k)
    ])

    # Solve Least Squares: min ||y - Phi * alpha||_2^2
    alpha, _, _, _ = np.linalg.lstsq(Phi, y, rcond=None)

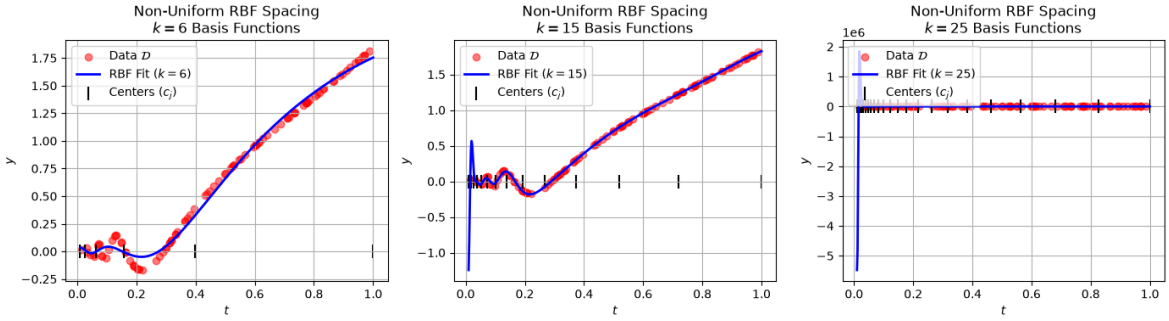
    # ------------------------------------------------------------------
    # 4. Evaluate Fitted Model on Fine Grid
    # ------------------------------------------------------------------
    Phi_fine = np.column_stack([
        np.exp(-((t_fine - centers[j]) ** 2) / (2 * sigmas[j] ** 2))
        for j in range(k)
    ])
    y_fine_hat = Phi_fine @ alpha

    # ------------------------------------------------------------------
    # 5. Plot Results and Highlight Center Density
    # ------------------------------------------------------------------
    plt.subplot(1, len(k_values), idx + 1)
    plt.scatter(t, y, color="red", alpha=0.5, label=r"Data $\mathcal{D}$"
    plt.plot(t_fine, y_fine_hat, color="blue", linewidth=2, label=f"RBF F

    # Visual mark for non-uniform RBF centers along x-axis
    plt.scatter(
        centers,
        np.zeros_like(centers),
        color="black",
        marker="|",
        s=120,
        linewidths=1.5,
        label="Centers ($c_j$)"
    )

    plt.title(f"Non-Uniform RBF Spacing\n$k = {k}$ Basis Functions")
    plt.xlabel("$t$")
    plt.ylabel("$y$")
    plt.legend(loc="upper left")
    plt.grid(True)

plt.tight_layout()
plt.show()
```



Triangular Basis Functions (Tent Functions)

A **Triangular Basis Function** (also known as a linear B-spline or tent function) is a piecewise linear, locally supported basis function. It peaks at a designated center point c_j with an activation value of 1.0 and decays linearly to 0.0 at a distance determined by its half-width w_j. Outside this interval, its value is strictly 0.0.

The mathematical definition for the j-th triangular basis function $f_j(t)$ is given by:

$$ f_j(t) = \max\left(0,\, 1 - \frac{|t - c_j|}{w_j}\right) $$

Alternatively, expressed as a piecewise conditional function over the interval $[c_j - w_j,\, c_j + w_j]$:

$$ f_j(t) = \begin{cases} 1 - \dfrac{c_j - t}{w_j}, & c_j - w_j \le t \le c_j \\[2ex] 1 - \dfrac{t - c_j}{w_j}, & c_j < t \le c_j + w_j \\[2ex] 0, & \text{otherwise} \end{cases} $$

Key Parameters

- **Center ($c_j \in \mathbb{R}$):** The knot location where the basis function attains its peak value of 1.0.
- **Half-Width ($w_j > 0$):** The distance from the center c_j to the boundary where the function reaches 0.0.

Model Approximation via Triangular Basis Combination

Using k triangular basis functions to approximate an unknown function $\hat{y} = f(t)$, the prediction model is expressed as a linear combination:

$$ \hat{y}(t) = \sum_{j=1}^{k} \alpha_j f_j(t) = \sum_{j=1}^{k} \alpha_j \max\left(0,\, 1 - \frac{|t - c_j|}{w_j}\right) $$

In matrix form for a dataset $\mathcal{D} = \{(t_i, y_i)\}_{i=1}^N$, the prediction vector $\hat{\mathbf{y}} \in \mathbb{R}^N$ is given by:

$$ \hat{\mathbf{y}} = \mathbf{\Phi} \mathbf{\alpha} $$

where the design matrix $\mathbf{\Phi} \in \mathbb{R}^{N \times k}$ is defined as:

$$ \mathbf{\Phi} = \begin{bmatrix} f_1(t_1) & f_2(t_1) & \cdots & f_k(t_1) \\ f_1(t_2) & f_2(t_2) & \cdots & f_k(t_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(t_N) & f_2(t_N) &

\cdots & f_k(t_N) \end{bmatrix} $$

The optimal model parameters $\mathbf{\alpha}^* = [\alpha_1, \alpha_2, \ldots, \alpha_k]^\top \in \mathbb{R}^k$ are determined by solving the Least Squares problem:

$$ \mathbf{\alpha}^* = \arg\min_{\mathbf{\alpha}} \|\mathbf{y} - \mathbf{\Phi}\mathbf{\alpha}\|_2^2 = (\mathbf{\Phi}^\top \mathbf{\Phi})^{-1} \mathbf{\Phi}^\top \mathbf{y} $$

In [21]:

```
import numpy as np
import matplotlib.pyplot as plt

# 1. Define evaluation domain [0, 1]
t = np.linspace(0, 1, 500)

# 2. Define 5 knot centers across [0, 1]
centers = np.array([0.0, 0.25, 0.5, 0.75, 1.0])
k = len(centers)

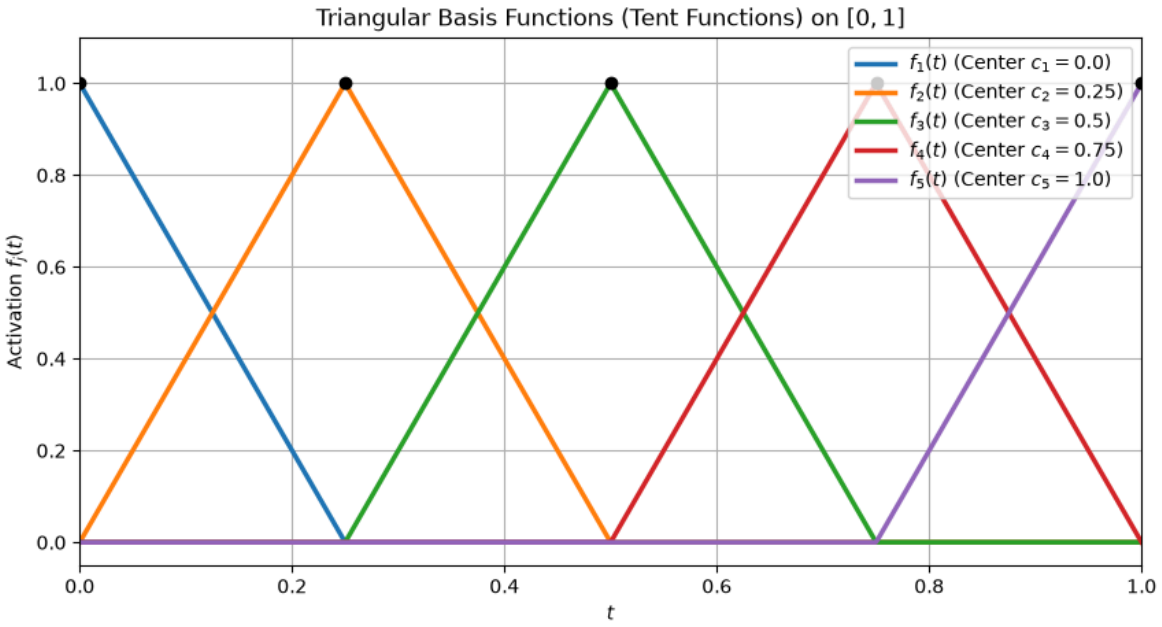
# Triangular basis function definition: f_j(t) = max(0, 1 - |t - c_j| / w
def triangular_basis(t_vec, c, width):
    return np.maximum(0.0, 1.0 - np.abs(t_vec - c) / width)

plt.figure(figsize=(10, 5))

# 3. Plot each triangular basis function
for j in range(k):
    # Determine local half-width based on neighboring centers
    if j == 0:
        width = centers[1] - centers[0]
    elif j == k - 1:
        width = centers[-1] - centers[-2]
    else:
        width = centers[j] - centers[j-1]

    f_j = triangular_basis(t, centers[j], width)
    plt.plot(t, f_j, linewidth=2.5, label=f'$f_{j+1}(t)$ (Center $c_{j+1}
    plt.scatter(centers[j], 1.0, color='black', zorder=5)  # Mark center

plt.title('Triangular Basis Functions (Tent Functions) on $[0, 1]$', font
plt.xlabel('$t$')
plt.ylabel('Activation $f_j(t)$')
plt.ylim(-0.05, 1.1)
plt.xlim(0, 1)
plt.legend(loc='upper right')
plt.grid(True)
plt.show()
```



In [20]:

```python
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------------------------------
# 1. Generate Training Data D = {(t_i, y_i)} over (0, 1]
# ------------------------------------------------------------------------
np.random.seed(42)
N = 100
t = np.random.uniform(0.01, 1.0, N)
y = t * np.sin(1 / t) + t**2

# Evaluation grid for visualization
t_fine = np.linspace(0.01, 1.0, 500)

# Helper function to compute Triangular Basis output
def triangular_basis(t_vec, center, width):
    return np.maximum(0.0, 1.0 - np.abs(t_vec - center) / width)

# ------------------------------------------------------------------------
# 2. Fit and Plot using Triangular Basis Functions
# ------------------------------------------------------------------------
k_values = [6, 15, 30]

plt.figure(figsize=(14, 4))

for idx, k in enumerate(k_values):
    # Log-spaced centers (dense near 0, sparse near 1)
    centers = np.logspace(np.log10(0.01), np.log10(1.0), k)

    # Half-width set to reach the adjacent center for overlap
    widths = np.zeros(k)
    if k > 1:
        widths[0] = centers[1] - centers[0]
        widths[-1] = centers[-1] - centers[-2]
        widths[1:-1] = np.maximum(centers[1:-1] - centers[:-2], centers[2
    else:
        widths = np.array([0.5])

    # 3. Construct Design Matrix Phi in R^(N x k)
    Phi = np.column_stack([
```

```
            triangular_basis(t, centers[j], widths[j])
            for j in range(k)
        ])

        # 4. Solve Least Squares: min ||y - Phi * alpha||_2^2
        alpha, _, _, _ = np.linalg.lstsq(Phi, y, rcond=None)

        # 5. Evaluate on fine grid
        Phi_fine = np.column_stack([
            triangular_basis(t_fine, centers[j], widths[j])
            for j in range(k)
        ])
        y_fine_hat = Phi_fine @ alpha

        # 6. Plot Results
        plt.subplot(1, len(k_values), idx + 1)
        plt.scatter(t, y, color="red", alpha=0.5, label=r"Data $\mathcal{D}$"
        plt.plot(t_fine, y_fine_hat, color="blue", linewidth=2, label=f"Trian

        # Mark centers along x-axis
        plt.scatter(
            centers,
            np.zeros_like(centers),
            color="black",
            marker="|",
            s=120,
            linewidths=1.5,
            label="Centers ($c_j$)"
        )

        plt.title(f"Triangular Basis Model\n$k = {k}$ Basis Functions")
        plt.xlabel("$t$")
        plt.ylabel("$y$")
        plt.legend(loc="upper left")
        plt.grid(True)

plt.tight_layout()
plt.show()
```

